MODULE 23 · DOMAIN 3 - CLOUD NATIVE APPLICATION DELIVERY

Packaging: Helm and Kustomize

Charts, values and releases; bases and overlays; and when to use each.

DOMAIN 3 of 4 EXAM WEIGHT 16% MODULE 23 of 35

admin@sthithapragna.com

Helm **templates** YAML and packages it. Kustomize **patches** plain YAML. Both end as ordinary manifests.

The problem both tools solve

- **One app, many environments:** development, staging, production.
- **Small differences:** replica counts, image tags, hostnames, resources, namespaces.
- **Copying the YAML per environment** drifts: a fix lands in one copy and not the others.
- **The goal:** one source of truth, plus a small, explicit list of differences.

> ON THE EXAM
>
> Copy-and-edit YAML per environment is the anti-pattern both tools replace.

Helm: the package manager for Kubernetes

- **A chart** is a package: templates plus default values. A graduated CNCF project.
- **Values** are the inputs: your settings override the chart's defaults.
- **A release** is one installed copy of a chart, with a numbered revision history.
- **Charts are shared** through repositories and OCI registries; Artifact Hub helps you find them.

> ON THE EXAM
>
> Chart: the package. Values: the settings. Release: an installed instance.

Inside a chart

FILE OR FOLDER	HOLDS
`Chart.yaml`	Name, chart version, and appVersion of the software inside
`values.yaml`	Default values for every setting
`templates/`	Manifests with placeholders, rendered using the values
`charts/`	Other charts this one depends on
`values.schema.json`	Optional: rules that validate supplied values

ON THE EXAM

version is the chart's own version; appVersion is the version of the application it installs.

A template, and the values that fill it

```
# templates/deployment.yaml (excerpt)
spec:
  replicas: {{ .Values.replicaCount }}
  template:
    spec:
      containers:
      - name: api
        image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
---
# values.yaml
replicaCount: 2
image: {repository: quayside/booking-api, tag: 2.4.1}
```

3 A placeholder, filled from the values when the chart is rendered.

11-12 The chart's defaults. Anything you supply overrides them.

The Helm commands you will see

```
$ helm repo add quayside https://charts.quayside.example
$ helm install booking quayside/booking-api -n quayside -f prod-values.yaml
$ helm upgrade booking quayside/booking-api -n quayside --set replicaCount=6
$ helm history booking -n quayside
$ helm rollback booking 3 -n quayside
$ helm uninstall booking -n quayside
```

2 Release name, then chart, then your values file.

3 New values or a new chart version: a new revision.

5 Back to revision 3, recorded as a new revision.

More commands worth knowing

COMMAND OR FLAG	DOES
`helm template`	Renders the manifests locally. Installs nothing
`helm show values`	Prints a chart's defaults, to see what you can set
`helm lint`	Checks a chart for problems
`helm list`	Lists releases in a namespace
`helm upgrade --install`	Installs if missing, upgrades if present
`--rollback-on-failure`	Rolls back automatically if the upgrade fails. Formerly --atomic

ON THE EXAM

'See the exact manifests without installing': helm template.

Which value wins?

- **Lowest:** the chart's own values.yaml.
- **Then:** each -f values file, in order; a later file beats an earlier one.
- **Highest:** --set on the command line.
- **Best practice:** keep environment values in files in Git; keep --set for rare one-offs.

> ON THE EXAM
>
> Chart defaults, then files in order, then --set. The most specific, latest source wins.

Releases and their history

- **Every install, upgrade and rollback** creates a new numbered revision.
- **Revisions are stored as Secrets** in the release's namespace.
- **No server component:** the old in-cluster Tiller was removed back in Helm 3.
- helm uninstall deletes the release's resources, but not claims kept by a StatefulSet.

> ON THE EXAM
>
> A Helm rollback, like a Deployment rollback, is recorded as a new revision.

Currency check: Helm 4

- **Helm 4 went GA on 12 November 2025,** the first major version in six years.
- **Existing charts work:** apiVersion v2 charts and Helm 3 releases carry over unchanged.
- **Renamed flags:** --atomic is now --rollback-on-failure; --force is now --force-replace.
- **New releases use server-side apply** by default.
- **Helm 3:** final feature release 9 September 2026; security fixes end 10 February 2027.

> ON THE EXAM
>
> Old flags still work, with a deprecation warning. Expect either name in questions.

Kustomize keeps plain YAML, and describes each environment as patches on a shared base.

Kustomize

- **No templates, no new language.** Just YAML and a kustomization.yaml file.
- **A base** holds the common manifests; **overlays** hold each environment's changes.
- **Built into kubectl:** kubectl apply -k, and kubectl kustomize to preview.
- **Common changes:** patches, image tags, namespace, name prefixes, labels, replicas, generated ConfigMaps.

> ON THE EXAM
>
> Kustomize needs no install: kubectl apply -k folder/ builds and applies it.

A base and two overlays

```
booking-api/
  base/         deployment.yaml  service.yaml  kustomization.yaml
  overlays/
    staging/    kustomization.yaml
    prod/       kustomization.yaml  replicas-patch.yaml

# overlays/prod/kustomization.yaml
resources: [../../base]
namespace: quayside
images: [{name: quayside/booking-api, newTag: 2.4.1}]
patches: [{path: replicas-patch.yaml}]
```

8 Start from the shared base.

10-11 Production's differences: its image tag and a replica patch.

THE TAKEAWAY

kubectl apply -k overlays/prod builds base plus production changes, and applies them.

configMapGenerator: config changes that roll out

- **Kustomize generates the ConfigMap** from files or literals.
- **It appends a hash of the contents** to the name: booking-config-7g2k9m.
- **Change the contents, and the name changes,** so the Pod template changes, so a rollout happens.
- Solves module 9's problem: editing a ConfigMap alone never restarts Pods.

> ON THE EXAM
>
> Hashed names turn every config change into a normal, reversible rollout.

Helm versus Kustomize

HELM

Templates and packages

- Templates plus values
- Packages to share and version
- Release history and rollback
- Best for installing others' software

KUSTOMIZE

Patches on plain YAML

- Plain YAML plus patches
- No packaging of its own
- Built into kubectl
- Best for your own apps' environments

THE TIEBREAKER

Installing third-party software: Helm. Your own app across environments, plain YAML: Kustomize. Many teams use both.

Six packaging decisions. For each, ask: **whose software**, and **which differences**?

> Quayside wants to install a widely used open-source monitoring stack, with sensible defaults and a few settings changed.
>
> Later it must upgrade to new versions, and roll back if an upgrade goes wrong.

What is the most practical approach?

A Install the project's Helm chart with a values file

B Write all of the manifests by hand from scratch

C A Kustomize overlay with no base to start from

D kubectl run for each component of the stack

◆ PAUSE AND DECIDE

What is the most practical approach?

A Install the project's Helm chart with a values file
Vendor defaults, your overrides, versioned upgrades and rollback.

B Write all of the manifests by hand from scratch
Huge effort, and you own every future upgrade.

C A Kustomize overlay with no base to start from
An overlay needs a base, which you would have to write.

D kubectl run for each component of the stack
Creates bare Pods: no Services, no upgrades.

THE SIGNAL

Someone else's software, upgrades and rollback: a Helm chart.

> A chart's values.yaml sets replicaCount: 2. The team installs it with -f prod-values.yaml, which sets replicaCount: 4.
>
> They also pass --set replicaCount=6 on the command line.

How many replicas are created?

A 2, from the chart's own defaults

B 4, from the production values file

C 6, from the --set flag

D None: Helm rejects the conflict

◆ PAUSE AND DECIDE

How many replicas are created?

A 2, from the chart's own defaults

Chart defaults have the lowest priority.

B 4, from the production values file

Files beat defaults, but --set beats files.

C 6, from the --set flag

--set has the highest priority.

D None: Helm rejects the conflict

Overrides are normal; nothing conflicts.

THE SIGNAL

Defaults, then -f files, then --set. --set wins.

> Quayside's own booking API runs in development, staging and production. The only differences are replica counts, image tags and namespaces.
>
> The team wants plain YAML, no templating language, and nothing extra to install beyond kubectl.

Which approach fits?

A. Write a Helm chart with a values file per environment

B. A Kustomize base with one overlay per environment

C. Copy the YAML folder once for each environment

D. One set of YAML, edited by hand before each deploy

◆ PAUSE AND DECIDE

Which approach fits?

A Write a Helm chart with a values file per environment

Works, but adds a templating language they wanted to avoid.

B A Kustomize base with one overlay per environment

Plain YAML, small patches, built into kubectl.

C Copy the YAML folder once for each environment

Copies drift: fixes land in one and not the others.

D One set of YAML, edited by hand before each deploy

Error-prone, and nothing records the differences.

THE SIGNAL

Own app, plain YAML, small differences, kubectl only: Kustomize.

> A Helm 4 upgrade of the booking API failed halfway: the new Pods crash and the release is left in a broken state.
>
> For future upgrades, the team wants Helm to roll back automatically whenever an upgrade fails.

Which flag should the pipeline add?

A --force-replace

B helm template instead of upgrade

C --dry-run

D --rollback-on-failure

◆ PAUSE AND DECIDE

Which flag should the pipeline add?

A --force-replace

Replaces resources forcibly; it does not roll back.

B helm template instead of upgrade

Renders manifests; it deploys nothing at all.

C --dry-run

Previews the upgrade without doing it.

D --rollback-on-failure

Formerly --atomic: roll back if the upgrade fails.

THE SIGNAL

Automatic rollback on a failed upgrade: --rollback-on-failure, once called --atomic.

> Before merging a change, a reviewer wants to see the exact manifests a chart will produce with the team's production values.
>
> Nothing may be installed or changed in any cluster.

Which command fits?

A helm install with the production values

B helm history for the release

C helm template with the production values

D helm lint on the chart folder

◆ PAUSE AND DECIDE

Which command fits?

A helm install with the production values
Installs the release, which is not allowed.

B helm history for the release
Lists past revisions, not future manifests.

C helm template with the production values
Renders the manifests locally; installs nothing.

D helm lint on the chart folder
Reports problems, but is not a full manifest review.

THE SIGNAL

'Exact manifests, install nothing': helm template.

> The team uses a Kustomize configMapGenerator for the booking API's settings file. They change one setting and run kubectl apply -k.
>
> Every booking API Pod is replaced by a rolling update, although nobody touched the Deployment.

Why did the Pods roll?

A Its name has a content hash, so the Pod template changed

B The kubelet restarts Pods whenever a ConfigMap changes

C Kustomize deletes Pods directly after every apply

D ConfigMaps are immutable, so all Pods were recreated

◆ PAUSE AND DECIDE

Why did the Pods roll?

A Its name has a content hash, so the Pod template changed

New contents, new name, new template: a normal rollout.

B The kubelet restarts Pods whenever a ConfigMap changes

Module 9: ConfigMap edits alone never restart Pods.

C Kustomize deletes Pods directly after every apply

Kustomize only builds manifests; it never deletes Pods.

D ConfigMaps are immutable, so all Pods were recreated

Immutability is opt-in, and would not cause a rollout.

THE SIGNAL

configMapGenerator hashes the contents into the name, so config changes roll out.

Module 23 in five lines

- **Helm:** charts (templates + values), releases with revisions and rollback. CNCF graduated.
- **Value priority:** chart defaults, then -f files in order, then --set.
- **helm template** renders without installing; **--rollback-on-failure** was --atomic.
- **Helm 4** GA November 2025; Helm 3 security fixes end February 2027.
- **Kustomize:** plain YAML, base plus overlays, kubectl apply -k; generators hash names.

Charts and overlays live in Git. Next: GitOps, where Git itself drives the cluster.

admin@sthithapragna.com